

SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-012: Global Dark Mode and Light Mode for the Full Student Management System

Field	Details
Change Request ID	CR-012
Change Request Title	Global Dark Mode and Light Mode for the Full Student Management System
Student Name	MSF.Shimla
Student ID	JF/ICT/24/14
Implementation Status	Completed
Date	10 October 2026

1. Change Request Summary

1.1 Business Requirement

The Student Management System required a consistent Dark Mode and Light Mode feature to improve usability and provide users with a flexible interface.

The theme selection must be shared across the entire system. When a user activates Dark Mode on the Login page, the other applicable pages must also display Dark Mode. Similarly, when the user switches to Light Mode from an Admin or Student Dashboard, the theme must change throughout the system.

The selected theme must remain consistent during page navigation, browser refresh, logout, and subsequent login.

1.2 Existing Behaviour

Before implementation:

- The application did not have a consistently connected theme toggle across all applicable pages.
- Theme controls were not consistently available on the Login and dashboard interfaces.
- Theme changes were not managed through a single shared application state.
- Page-specific CSS could prevent all UI elements from displaying the selected theme correctly.
- A consistent theme preference across the complete application was required.

1.3 Requested Change

A reusable Dark Mode / Light Mode toggle was required throughout the Student Management System.

The feature must:

- Display a theme toggle on the Login page.
- Display a theme toggle on the Admin Dashboard.
- Display a theme toggle on the Student Dashboard.
- Apply the selected theme throughout the application.
- Allow users to switch themes without reloading the page.
- Store the selected theme in localStorage.
- Preserve the selected theme after navigation and browser refresh.
- Apply the same theme to forms, cards, tables, navigation, buttons, and modals.
- Maintain existing authentication, authorization, API, and CRUD functionality.
- Keep the implementation within the frontend without requiring database changes.

2. Implementation

2.1 Implementation Summary

CR-012 introduces a shared theme-management mechanism for the Student Management System.

A React Context is used to maintain the selected theme and provide a common state to the application components. The ThemeProvider wraps the application so that all pages can access the same theme state.

A reusable ThemeToggle component provides the Dark Mode and Light Mode button. The component is designed to be used across different pages without duplicating the theme-switching logic.

The selected theme is stored in browser localStorage. A CSS theme attribute is applied to the document root so that global and page-specific styles can respond to the selected theme.

The implementation also includes Login page integration and styling adjustments for the theme toggle. Existing login handling, dashboard navigation, and student-management operations are intended to remain unchanged.

2.2 Affected Files

File	Change	Purpose
frontend/src/context/ThemeContext.jsx	Added	Manages the shared theme state, toggle function, and localStorage persistence.
frontend/src/components/ThemeToggle.jsx	Added	Provides a reusable Dark Mode / Light Mode button.
frontend/src/main.jsx	Modified	Wraps the application with ThemeProvider.
frontend/src/App.jsx	Existing	Continues rendering the application routes.
frontend/src/routes/AppRoutes.jsx	Verify	Ensures the existing routes render the correct pages.
frontend/src/components/Navbar.jsx	Modified / Verify	Provides access to the theme toggle on the applicable navigation interface.
frontend/src/components/LoginInfo.jsx	Modified	Contains the Login information section and Back to Home navigation link.
frontend/src/components/LoginFormSection.jsx	Verify	Ensures the existing login form remains functional and readable in both themes.
frontend/src/pages/LandingPage.jsx	Modified / Verify	Applies the global theme to the landing page.
frontend/src/pages/LoginPage.jsx	Modified	Integrates ThemeToggle into the Login page while preserving login handling.
frontend/src/pages/DashboardPage.jsx	Modified / Verify	Integrates the shared theme toggle into the Admin Dashboard.
frontend/src/pages/StudentDashboardPage.jsx	Modified / Verify	Integrates the shared theme toggle into the Student Dashboard.
frontend/src/styles/global.css	Modified	Defines common theme variables and reusable component styling.
frontend/src/styles/landing.css	Modified / Verify	Supports light and dark styling on the landing page.

frontend/src/styles/login.css	Modified	Styles the Login page and positions the theme toggle.
frontend/src/styles/dashboard.css	Modified / Verify	Supports theme styling for the Admin Dashboard.
frontend/src/styles/studentDashboard.css	Modified / Verify	Supports theme styling for the Student Dashboard.

2.3 Technical Changes

A. Shared Theme Context

The ThemeContext component manages the selected theme using React state.

The shared state allows the Login page, Admin Dashboard, Student Dashboard, and other applicable components to use the same theme preference.

The context exposes the current theme and a toggle function to the components that consume it.

B. Reusable ThemeToggle Component

The ThemeToggle component provides a common interface for changing the theme.

The button displays the appropriate mode label and icon based on the currently selected theme.

The same component can be reused on different pages to prevent inconsistent implementations.

C. Global Theme State

The theme is applied using the document root's data-theme attribute.

The application uses the following theme values:

- light — Light Mode.
- dark — Dark Mode.

The CSS styles use theme variables to control backgrounds, text, borders, input fields, and other UI elements.

D. localStorage Persistence

The selected theme is stored using the following key:

app-theme

The application reads the saved preference when initializing the theme state.

After a theme change, the selected value is saved to localStorage. This allows the preference to remain available after a browser refresh or page navigation.

The theme preference contains only a non-sensitive UI setting.

E. Global Theme Synchronization

All applicable pages use the shared theme state rather than maintaining independent theme selections.

For example:

1. The user activates Dark Mode on the Login page.
2. The shared theme state changes to dark.
3. The document root receives the dark theme attribute.
4. Components and CSS rules respond to the updated theme.
5. The Admin Dashboard and Student Dashboard use the same selected theme.

The same process applies when switching from Dark Mode to Light Mode.

F. CSS Theme Styling

Theme variables are defined in global.css and reused in page-specific stylesheets.

The following UI elements require styling for both modes:

- Page backgrounds and text.
- Navigation bars and footers.
- Dashboard cards and panels.
- Login forms and input fields.
- Student tables and search interfaces.
- Buttons and action controls.
- Modals and dropdowns.
- Borders, labels, and secondary text.

Existing hardcoded CSS colours must be reviewed and updated where necessary to ensure consistent theme application.

G. Existing Functionality Preservation

The change is intended to affect presentation only.

The existing login function, role-based dashboard navigation, API calls, student records, and CRUD operations must remain unchanged.

No database migration or new theme-related API endpoint is required.

2.4 Implementation Logic

1. Create the ThemeContext component.
2. Initialize the theme from localStorage, using Light Mode as the default when no valid saved preference exists.
3. Wrap the application with ThemeProvider in main.jsx.
4. Create the reusable ThemeToggle component.
5. Integrate the toggle into the Login page and applicable dashboard interfaces.
6. Apply the selected theme to the document root using the data-theme attribute.
7. Save the selected theme using the app-theme localStorage key.
8. Update global and page-specific CSS to support both themes.
9. Verify that the same theme is used across all applicable pages.
10. Test page navigation, refresh, login, logout, and existing system functionality.

2.5 Error and Data Handling

The theme feature uses client-side state and localStorage. It does not require additional API requests or database operations.

The implementation must ensure that:

- The theme context is available to all components using the theme hook.
- Components using the context are rendered inside ThemeProvider.
- The theme toggle changes the selected theme without a page reload.
- Missing saved theme preferences result in the default Light Mode.
- Invalid saved theme values do not prevent the application from rendering.
- Existing authentication and authorization behaviour remains unchanged.
- Theme styling does not make text, controls, or error messages unreadable.

The Back to Home link on the Login page should also be verified against the application's actual Home route.

3. Testing and Verification

3.1 Testing Approach

The implementation requires manual testing to confirm the shared theme behaviour and ensure that existing functionality remains unaffected.

Testing Types:

- Functional testing.
- UI testing.
- Theme synchronization testing.
- Navigation testing.
- Persistence testing.
- Responsive UI testing.
- Regression testing.
- Authentication and authorization testing.
- Error and runtime testing.

Environment: VScode**Browser:** Google Chrome / Microsoft Edge / Firefox**Test Roles:** Unauthenticated User, Administrator, Student**Testing Status:** Pending final verification of the integrated application.**3.2 Test Cases**

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Open the Login page	Login page and theme toggle are displayed	verified	pass
TC-02	Activate Dark Mode on the Login page	The entire application uses Dark Mode	verified	pass
TC-03	Activate Light Mode on the Admin Dashboard	The entire application uses Light Mode	verified	pass
TC-04	Change the theme on the Student Dashboard	All applicable pages use the selected theme	verified	pass
TC-05	Navigate between Landing, Login, and Dashboard pages	The selected theme remains consistent	verified	pass
TC-06	Refresh the browser	The saved theme is restored	verified	pass
TC-07	Logout and log in again	The saved theme preference remains available	verified	pass

TC-08	Open Add/Edit Student forms or modals	Forms and modals use the selected theme	verified	pass
TC-09	Test the theme toggle on a mobile-sized screen	The button remains visible and usable	verified	pass
TC-10	Test valid and invalid login attempts	Existing login behaviour remains unchanged	verified	pass
TC-11	Test existing Add, Edit, Delete, and Search functions	Existing student-management operations continue to work	verified	pass
TC-12	Inspect the browser console and terminal	No new theme-related runtime or build errors occur	verified	pass
TC-13	Click Back to Home on the Login page	The application navigates to the correct Home route	verified	pass

3.3 Test Summary

Test Result	Count
Total Test Cases	13
Passed	13
Failed	0
Not Executed / Pending	0
Pass Rate	100%

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification Method	Status
AC-001	A Dark/Light toggle is visible on every required page	UI inspection	pass
AC-002	The Login page can change the global theme	Login theme test	pass
AC-003	The Admin Dashboard can change the global theme	Admin dashboard test	pass

AC-004	The Student Dashboard can change the global theme	Student dashboard test	pass
AC-005	Admin child pages use the selected theme	Admin navigation test	pass
AC-006	Student child pages use the selected theme	Student navigation test	pass
AC-007	Theme changes are shared across all applicable pages	Cross-page synchronization test	pass
AC-008	Theme switching occurs without a page reload	Functional test	pass
AC-009	The selected theme persists after browser refresh	Persistence test	pass
AC-010	The selected theme persists after logout and subsequent login	Authentication regression test	pass
AC-011	All major UI elements remain readable in both themes	UI inspection	pass
AC-012	Existing login and CRUD functionality remains operational	Regression testing	pass
AC-013	No theme-related runtime or build errors are introduced	Console and build verification	pass

5. Evidence

A maximum of two relevant screenshots should be attached, provided each screenshot demonstrates the required functionality clearly.

Evidence 01 – Login Page Theme Toggle

Related: AC-001, AC-002, AC-008

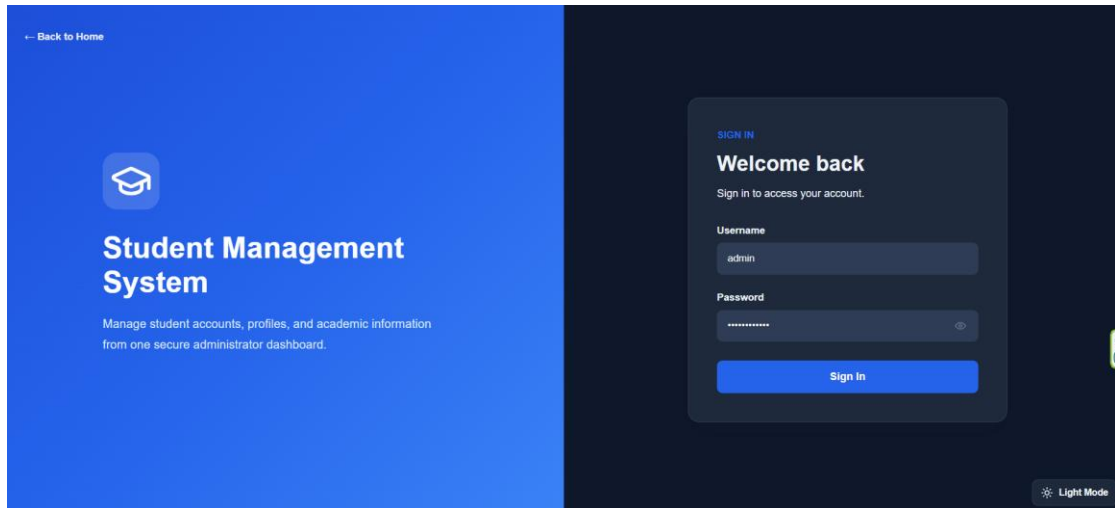


Figure 1: Login page displaying the Dark/Light Mode toggle and the selected theme.

The screenshot should demonstrate:

- The Login page.
- The visible theme toggle.
- The Login form and information section in the selected theme.

Evidence 02 – Global Theme Synchronization

Related: AC-003, AC-004, AC-005, AC-006, AC-007, AC-009

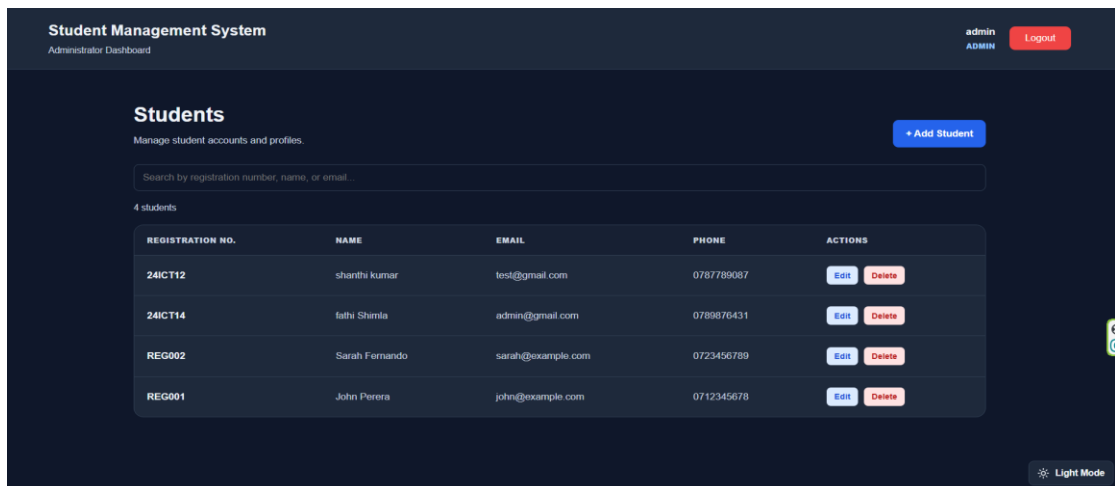


Figure 2: Admin or Student Dashboard displaying the selected theme after changing the theme on another page.

The screenshot should demonstrate the selected theme on the destination page. Where possible, include evidence of persistence after navigation or browser refresh.

6. Final Result

Item	Result
Requirements	completed
Implementation	completed
Testing	completed
Acceptance Criteria	completed
Existing CRUD	Verified
Final Status	Pending Testing and Acceptance

6.1 Final Summary

CR-012 introduces a shared Dark Mode and Light Mode feature for the Student Management System. The implementation uses a reusable ThemeToggle component and a shared React Context to manage the selected theme across applicable pages.

The selected theme is stored in localStorage using the app-theme key. The document root's data-theme attribute allows global and page-specific CSS styles to respond to theme changes.

The intended behaviour is to keep the Login page, Admin Dashboard, Student Dashboard, and other applicable pages synchronized with the same theme preference. The change is frontend-focused and does not require changes to authentication APIs, database structure, or student data.

The implementation also requires verification of theme persistence, UI readability, routing, and existing login and CRUD functionality. Final acceptance should be recorded only after the defined test cases have been executed successfully.

6.2 Issues / Limitations

The following items require final verification:

- The theme toggle must be visible and functional on all applicable pages.
- Global theme styling must be checked for hardcoded colours in existing stylesheets.
- The Back to Home navigation link must be tested against the actual route configuration.
- The project must compile and run without theme-related errors.
- Existing authentication and CRUD operations must be regression-tested.
- Actual test results and evidence screenshots must be added before final submission.

Student Name:	MSF.Shimla
Student ID:	JF/ICT/24/14
Date:	10 October 2026
Signature:	

~ END OF REPORT ~